

Tail Waste Fragmentation of KV Cache Under Virtual Memory Management

Group 3

Joshua Frank, Londy Jean, Beethoven Marhone, Kyle Merritt

Introduction

- Do we still need the vAttention custom driver that reduces page size from 2 MB to 64 KB, even at long context lengths?
- How can a user know in advance when 2 MB pages will cause significant tail waste for a given model and target context range?

Terminology

- Fragmentation- wasted or unusable memory due to allocation patterns
 - External – Free memory split into unusable spaces
 - Internal - Unused space inside allocated memory
- LLM Inference – Using a trained model to generate output tokens. Cycles through taking a previous token, running them through a transformer, compute next token probabilities
- KVCache – Stores immediate Key(K) and Value tensor from attention for each token layer allowing reuse of past computation

Terminology Continued

- Context Length – The total number of tokens processed so far (input + generated output) which determines how large the KV cache becomes
- Page/Block - A fixed-size unit of memory allocation used in paging systems; enables flexible allocation but can introduce internal fragmentation
- Advanced Attention Architectures – LLM attention architectures such as GQA and MLA attempt to reduce KV cache memory usage

Classical Solutions

- Contiguous Allocation

- Allocates one continuous block of memory per request. Memory must be physically adjacent and sized at allocation time
- Pro
 - Simple and fast memory access due to no indirection
- Con
 - Severe external fragmentation when request sizes are unpredictable.

- Memory Compaction

- Moves allocated memory blocks to merge scattered free space into once contiguous region
- Pro
 - Eliminates external fragmentation and restores large contiguous memory blocks
- Con
 - High overhead due to memory movement and pointer updates; not suitable for real time systems

Classical Solutions Continued

- slab / segregated-size allocator
 - Allocates memory from pools of fixed size objects, where each slab contains identical sized blocks
 - Pro
 - Reduces external fragmentation and improve allocation speed through object reuse
 - Con
 - Causes internal fragmentation and lacks flexibility for dynamically growing data
- PagedAttention
 - Divides KV Cache into fixed-size pages and uses a block table to map logical positions to noncontiguous physical memory
 - Pro
 - Eliminates the need for contiguous allocation and support dynamic KV cache growth
 - Con
 - Introduces kernel complexity and internal fragmentation due to fixed page size

Why Classical Methods Break for KV Cache

- **Dynamic KV growth** – Requests grow unpredictably during the inference
- **Real-time latency constraints** – Memory operations must be fast no expensive relocation
- **Mismatch with classical assumptions** - Traditional systems assume static sizes to allow memory movement
- The overall down fall is that these traditional systems leads to fragmentation, inefficiency, or performance overhead

Vattention

- VAttention addresses fragmentation by introducing a virtual memory abstraction for the KV cache
- Instead of requiring contiguous physical memory it allows the KV cache to grow dynamically using noncontiguous pages, while still appearing contiguous to the model.
- Avoids over allocation and eliminates the need for expensive memory movement

Attention Architectures

- Which are better according to theory?

Experiment

- From the paper "vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention" focuses on kernel efficiency and system throughput and does not analyze fragmentation vs context length
- Hypothesis
 - VAttention transforms fragmentation into a fixed overhead problem.
 - Fragmentation should decrease as context length increase because KV cache grows linearly with tokens while allocation overhead remains relatively constant, so the overhead becomes amortized over time

Tail-Waste Expressions

$$F_exact(C) = 100 * (\text{ceil}(C/T)T - C) / (\text{ceil}(C/T)T)$$

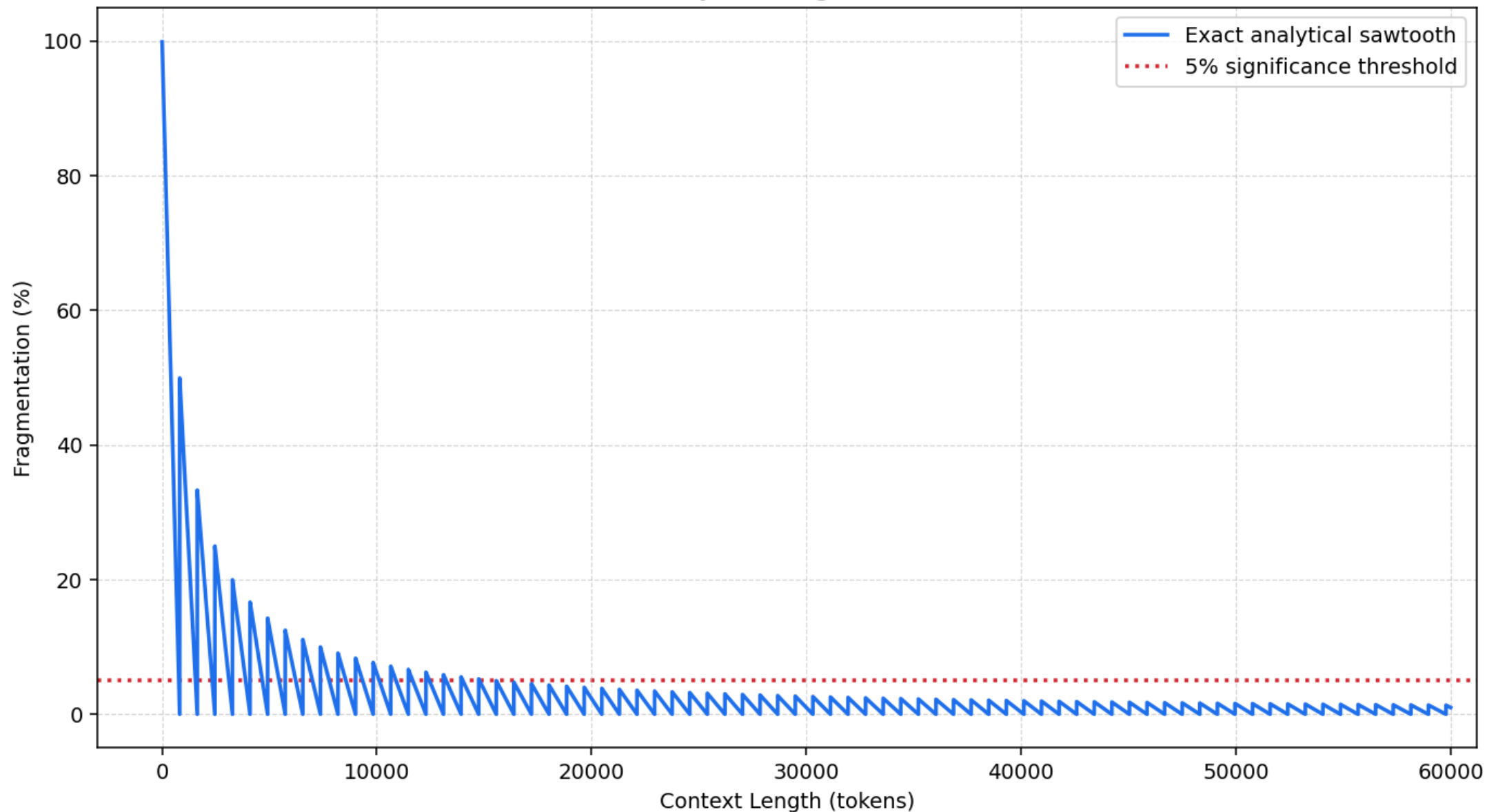
$$F_worst(C) \approx 100 * T / (C + T)$$

$$T_dense = \text{floor}(P / (H_kv,loc D b))$$

$$T_MLA = \text{floor}(P / ((r_kv + d_rope) b))$$

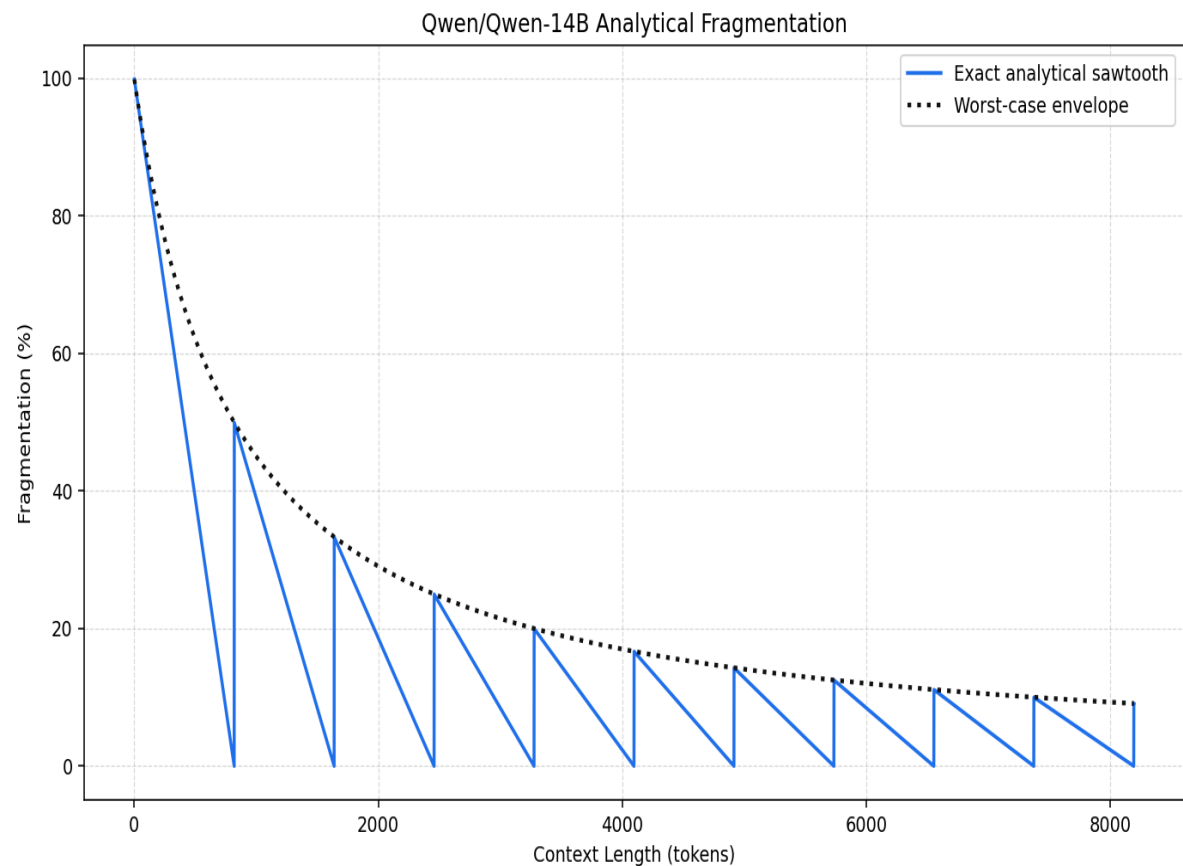
Synthetic MLA

Qwen-14B (MHA) Analytical Fragmentation to 60k Context

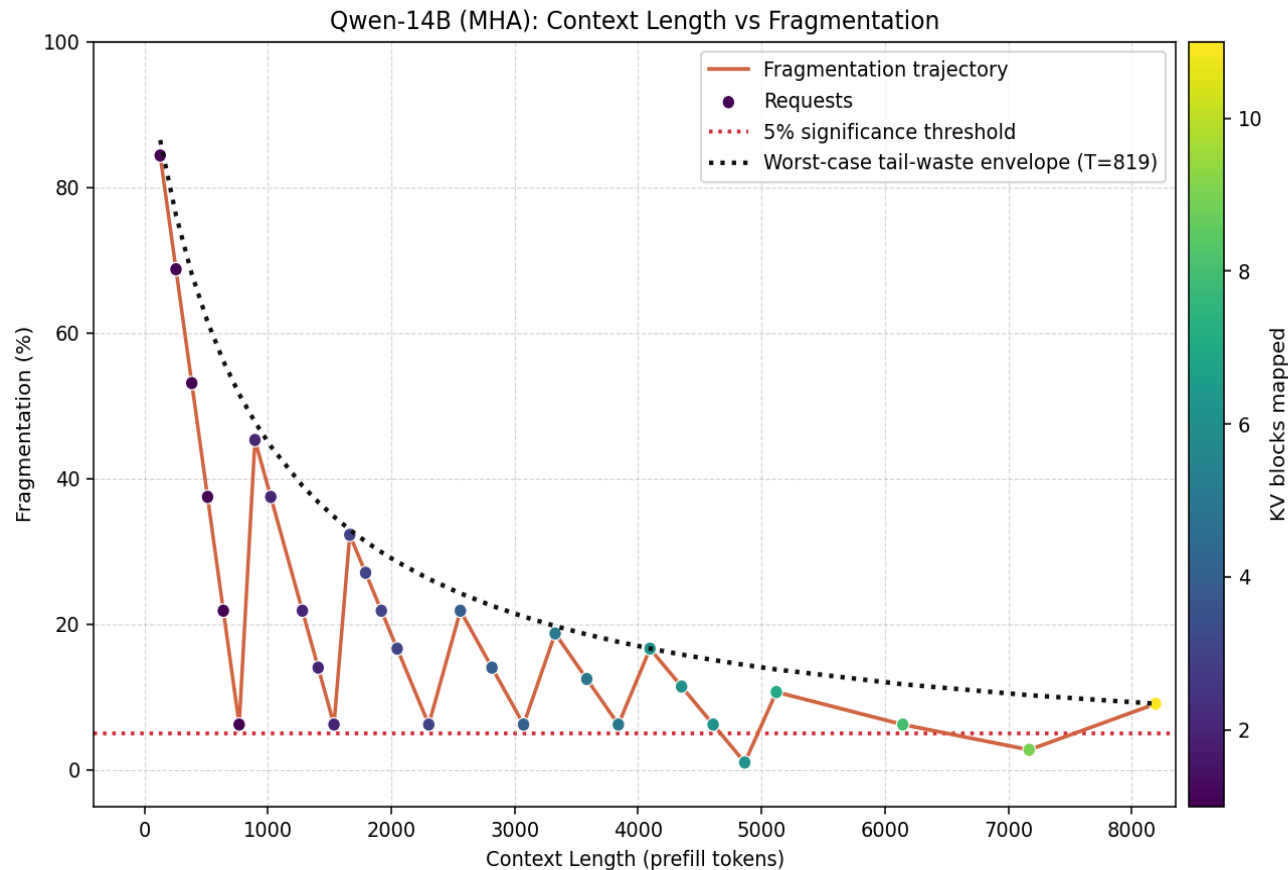


arch=dense_kv, T=819, page_buffer_token_bytes=2560

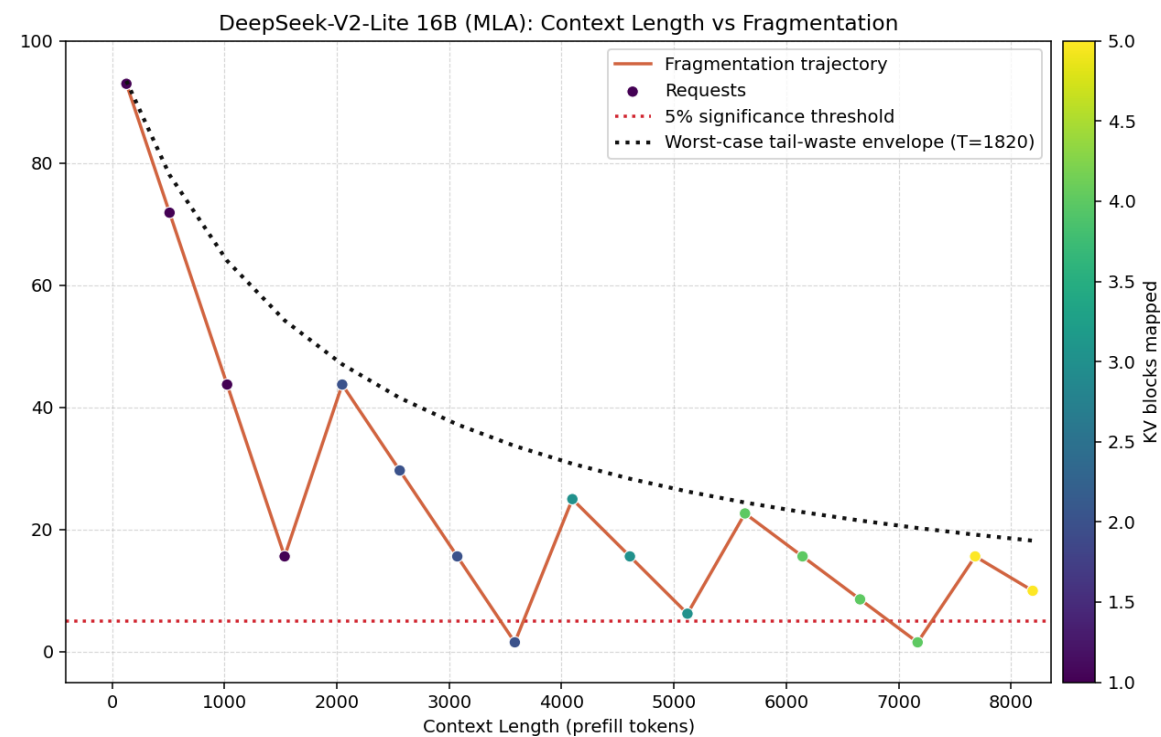
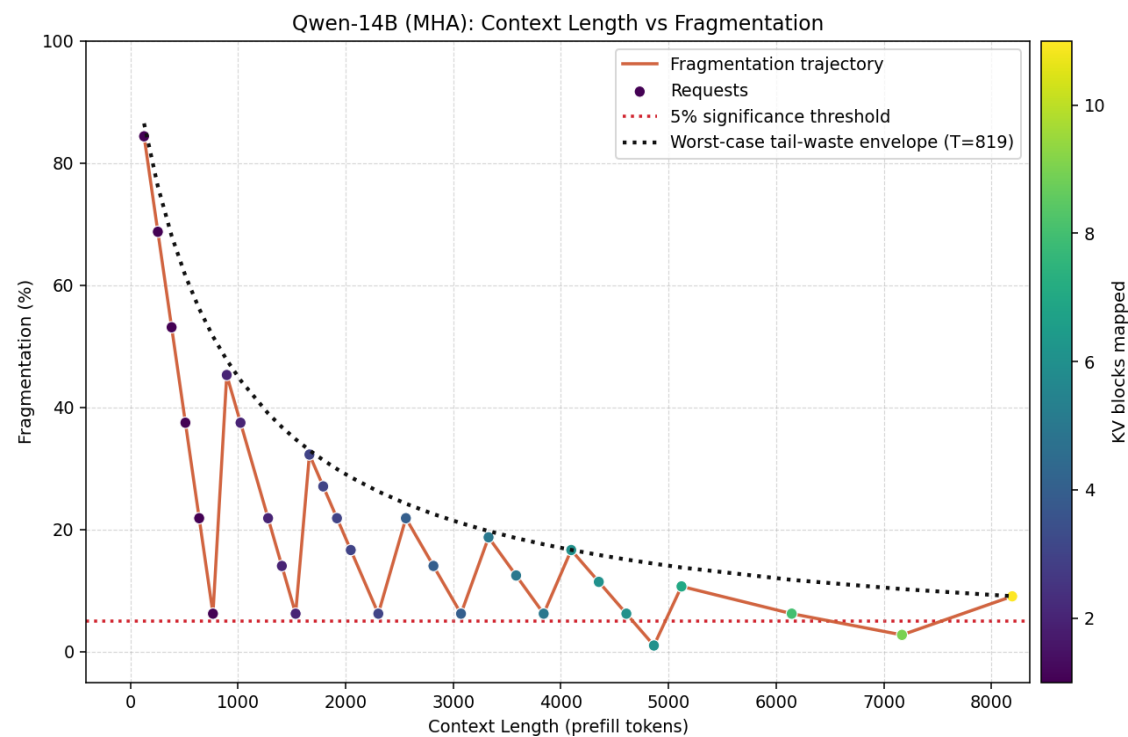
Analytical vs Empirical Fragmentation



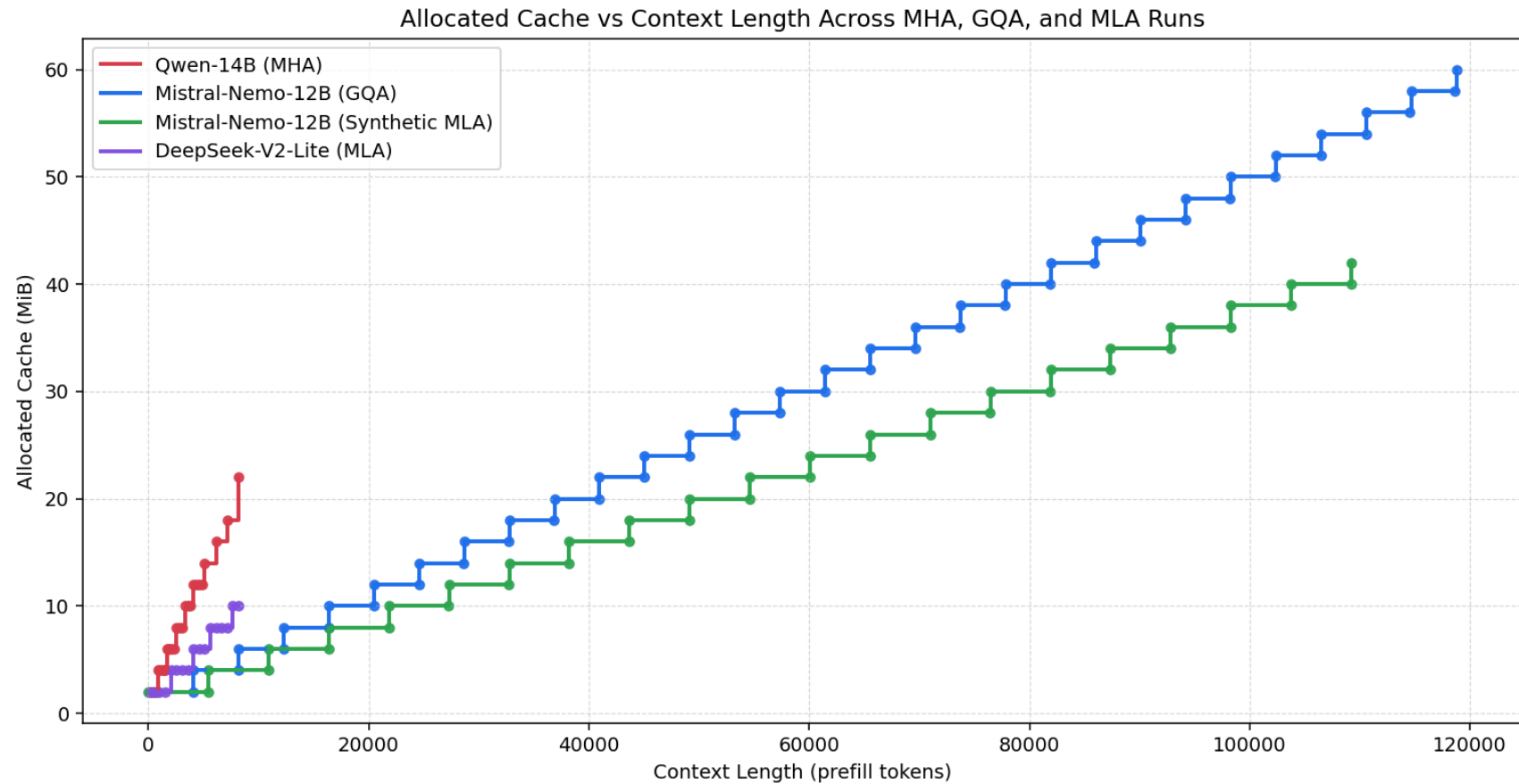
arch=dense_kv, T=819, page_buffer_token_bytes=2560



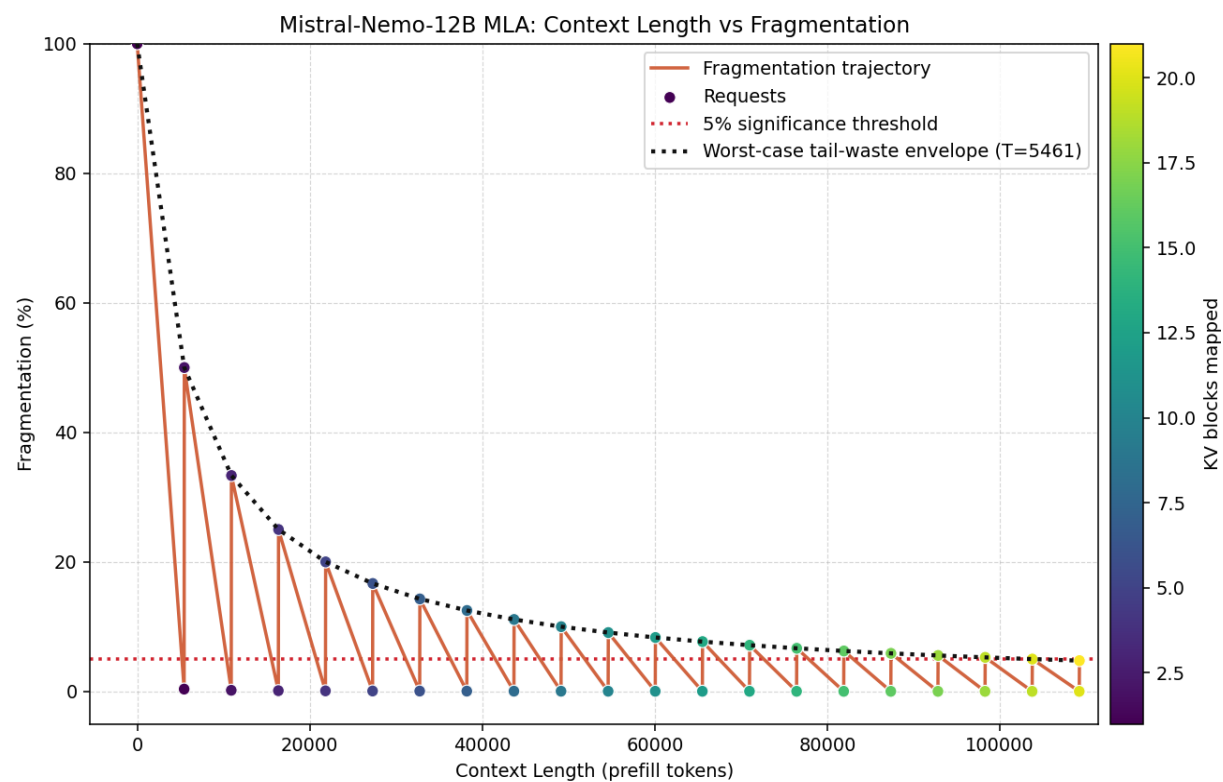
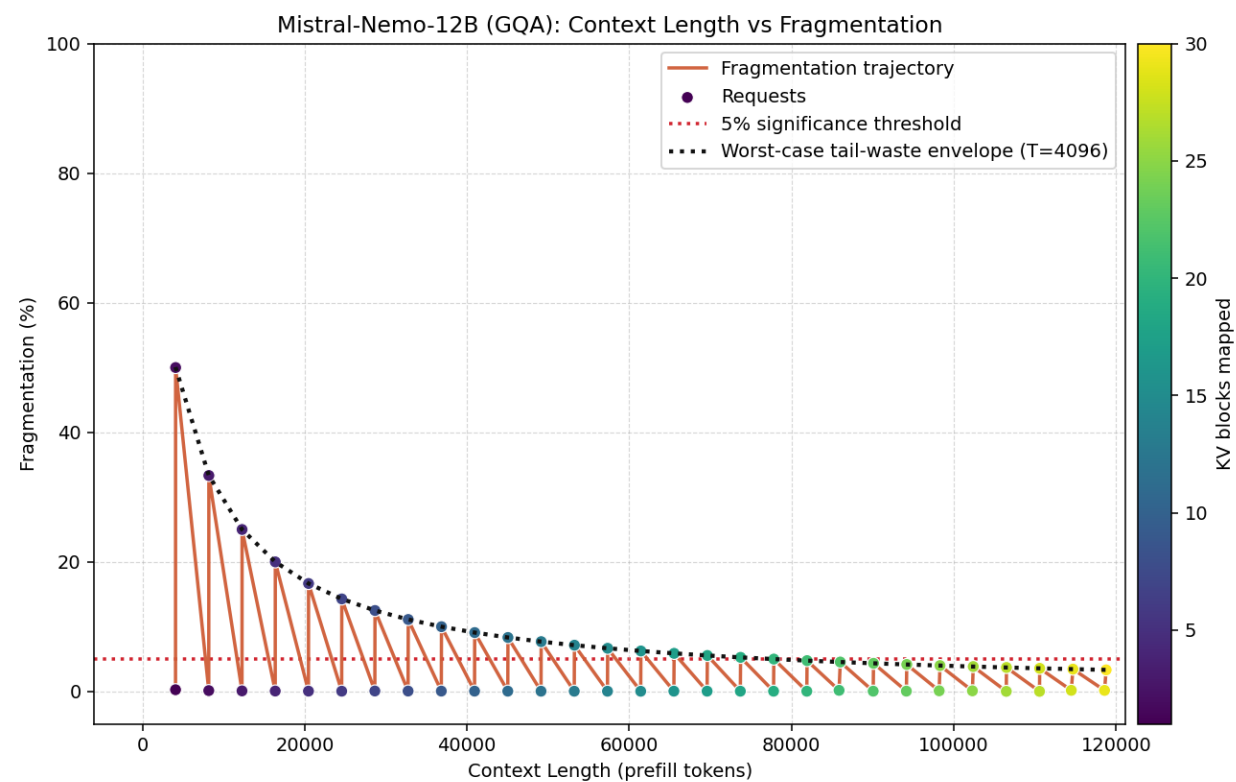
MHA vs MLA



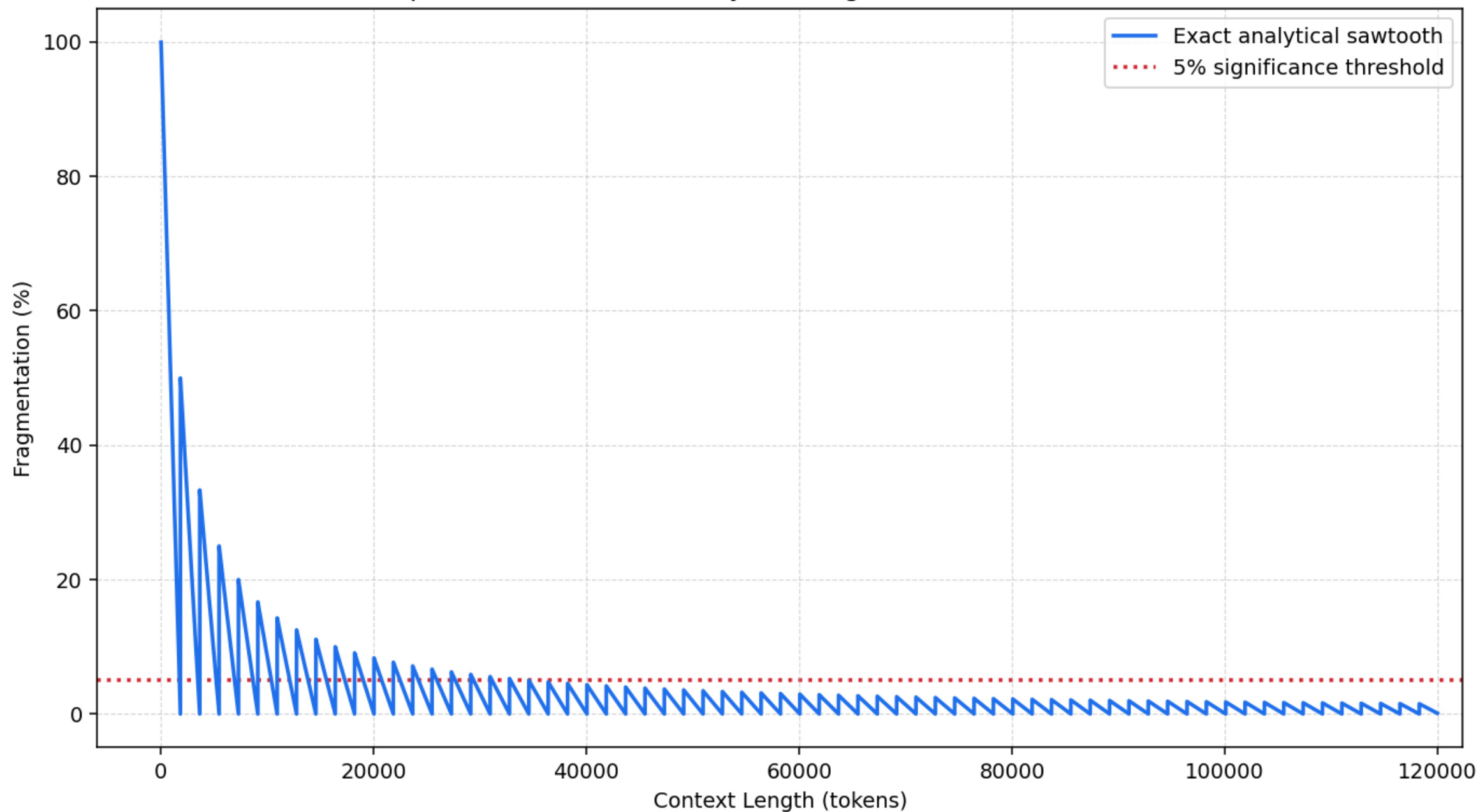
MHA vs GQA vs MLA



GQA vs MLA



DeepSeek-V2-Lite (MLA) Analytical Fragmentation to 120k Context



arch=mla, T=1820, page_buffer_token_bytes=1152

Conclusion

- Need for the custom driver depends on model architecture and context length.
- Our analytical model predicts fragmentation well and helps guide deployment decisions.